C/C++-Kurs für Kinder ab der 5. Klasse

Jürgen "George" Sawinski

10. August 2026

Copyright © 2026 Jürgen “George” Sawinski

VERÖFFENTLICHT VOM AUTOR

https://github.com/jsawinski/twin-c-course

Licensed under the MIT License

Copyright (c) 2026 Jürgen “George” Sawinski

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Dieser Kurs ist gedacht für Kinder ab Ende der Grundschule (ca. 5. Klasse), die Begeisterung für Mathematik und Computer mitbringen und den Drang haben, programmieren zu lernen.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

I Die Programmiersprache C 7

1 Ein paar Grundlagen 9

2 Erste Schritte in C 15

3 Etwas Physik 17

II Die Programmiersprache C++ 19

III Anhang 21

Vorwort

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten um ihre eigenen Spiele zu entwickeln zu können.

Die Mitschrift ist in erster Linie für Kinder gedacht, die den Kurs mit einer Begleitperson zusammen durchlaufen und dient als Leitfaden sowie als Nachschlagewerk für die behandelten Themen.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch KI-gestützte Werkzeuge (**DeepSeek** und **GitHub Copilot**) unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Github Copilot wurde beim Schreiben von LaTeX-Code und bei der Erstellung von C/C++-Codebeispielen eingesetzt. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

– Jürgen "George" Sawinski, 10. August 2026

Lizenz und Haftungsausschluss

Alle Inhalte dieses Kurses wurden vom Autor selbst erstellt und sind, soweit nicht anders gekennzeichnet, frei von urheberrechtlich geschützten Materialien Dritter. Die Beispiele und Übungen sind so gestaltet, dass sie die Lernziele unterstützen und den Kindern helfen, die Konzepte zu verstehen und anzuwenden.

Dieses Werk wird unter der **MIT-Lizenz** veröffentlicht. Das bedeutet: Ihr dürft es kopieren, verändern, weitergeben und für eigene Zwecke nutzen — auch kommerziell. Die einzige Bedingung ist, dass der Autor (Jürgen "George" Sawinski) in der Quellenangabe genannt wird.

Haftungsausschluss: Dieser Kurs wurde mit größter Sorgfalt erstellt. Dennoch übernimmt der Autor keine Gewähr für die Richtigkeit, Vollständigkeit oder Aktualität der Inhalte. Die Nutzung der Inhalte erfolgt auf eigene Verantwortung. Der Autor haftet nicht für Schäden, die aus der Nutzung oder der Unfähigkeit zur Nutzung der Inhalte entstehen.

Projekt-Webseite: `https://github.com/jsawinski/twin-c-course`

Teil I

Die Programmiersprache C

1
Ein paar Grundlagen

1.1 Das Binärsystem

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt "**Bit**" aneinandergereit. Am gebräuchlichsten dabei ist das "**Byte**" (siehe Abb. 1.1), das aus 8 Bit besteht.
Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

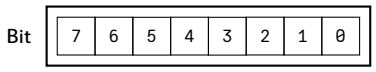


Abbildung 1.1: Ein Byte mit den Bit-Positionen 0 bis 7.

$$2^b$$

wobei b die Nummer des Bits ist. Das ist die sogenannte "Potenz", eine mathematische Operation[1] die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die "2" b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn Ihr genau hinschaut, dann merkt Ihr, dass, um in die nächste Zeile zu kommen, Ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).[2]

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 1.1).

[1] In der Schule habt Ihr wahrscheinlich den Begriff der Grundrechenarten (oder Grundrechnungsarten) kennengelernt. Der Begriff "Operation" FIXME

[2] Es gibt ein Ausnahme und das ist 0^0. Wenn Ihr diese Rechnung auf dem Computer ausführt, dann erhaltet Ihr als Ergebnis höchstwahrscheinlich auch die 1. Die Mathematik dahinter kommt allerdings nicht zu einem eindeutigen Ergebnis.

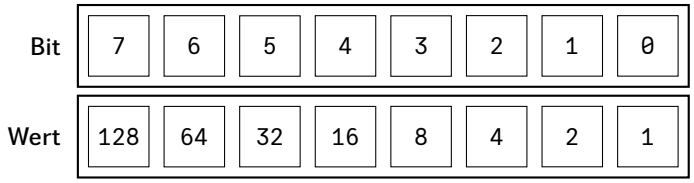


Abbildung 1.2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$\begin{aligned}
00000000 &\rightarrow 0\,,\\
00000011 &\rightarrow 3\,,\\
01011000 &\rightarrow 88\,,\\
11111111 &\rightarrow 255\,.
\end{aligned}$$

Das Addieren funktioniert nun genauso, wie Ihr es aus der Schule, in der natürlich das Zehnersystem verwendet wird, gelernt habt. Beispiel:

```
  00100111       39
+ 00000110     +  6
     11          1     (Übertrag)
 ---------      ---
  00101101       45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =     10·42
        ..|.|.|.
  00000000.|.|.|.
   00000000|.|.|.
    00001010.|.|.
     00000000|.|.
      00001010.|.
       00000000|.
        00001010.
         00000000
         1 1             (Übertrag)
 ---------------
       110100100         420
```

Wenn Ihr genau hinseht, dann bemerkt Ihr, dass wir zwei Bytes multipliziert haben, aber das Ergebnis benötigt 9 Bits, also ein Bit mehr als ein Byte. Das führt zu dem Problem, dass, wenn Ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte "1" links) abgeschnitten wird und das Ergebnis, dass der Computer euch liefert, 164 ist. Das gleiche passiert bei einem Byte, wenn Ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0 (in der Fachsprache als **Überlauf** bezeichnet).

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn 255 + 1 zum Ergebnis 0 hat, dann hat 0 – 1 als Ergebnis –1, binär also 11111111. Dies nennt man das "Zweierkomplement" (den Begriff müsst Ihr euch natürlich nicht merken). Wichtig ist: dass höchste

Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        -128
10000001        -127
...
11111111        -  1
00000000           0
00000001           1
...
01111111         127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
(-127)  10000001  ->  01111110
                     +00000001
                      --------
                      01111111     (127)
```

Das funktioniert auch umgekehrt, also aus einer negativen Zahl eine positive zu machen:

```
(127)   01111111  ->  10000000
                     +00000001
                      --------
                      10000001     (-127)
```

1.2 Logikgatter — die Bausteine des Prozessors

Bevor wir verstehen, wie Computer rechnen, müssen wir wissen, woraus sie gebaut sind. Das Herzstück eines Computers ist der Prozessor (die CPU). Er besteht aus vielen kleinen Schaltungen, die mit nur zwei Zuständen arbeiten: **0** und **1**.

Diese Schaltungen nennt man **Logikgatter**. Die wichtigsten sind:

UND-Gatter (AND)

Das UND-Gatter liefert nur dann eine 1, wenn beide Eingänge 1 sind.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

ODER-Gatter (OR)

Das ODER-Gatter liefert eine 1, wenn mindestens einer der Eingänge 1 ist.

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

XOR-Gatter (exklusives ODER)

Das XOR-Gatter liefert eine 1, wenn genau einer der Eingänge 1 ist.

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

NICHT-Gatter (NOT)

Das NOT-Gatter invertiert den Eingang: aus 0 wird 1, aus 1 wird 0.

A	NOT A
0	1
1	0

Aus diesen einfachen Gattern kann man alles bauen – sogar eine Addition!

1.3 Bitweise Operationen in C

In der Programmiersprache C könnt ihr diese Logikgatter direkt benutzen. Die Operatoren dafür heißen **bitweise Operatoren**, weil sie auf den einzelnen Bits einer Zahl arbeiten.

& (bitweises UND)

Arbeitet wie das UND-Gatter, aber auf jedem Bit einzeln.

```
  1101
& 1011
--------
  1001
```

| (bitweises ODER)

Arbeitet wie das ODER-Gatter, aber auf jedem Bit einzeln.

```
  1101
| 1011
--------
  1111
```

^ (bitweises XOR)

Arbeitet wie das XOR-Gatter, aber auf jedem Bit einzeln.

```
  1101
^ 1011
--------
  0110
```

~ (bitweise Negation)

Arbeitet wie das NOT-Gatter, aber auf jedem Bit einzeln.

```
 ~ 1101
--------
  0010
```

Wichtige Unterscheidung: In C gibt es auch **logische Operatoren** (`&&`, `||`, `!`). Die arbeiten nicht auf Bits, sondern auf *Wahrheitswerten* (true/false). Wir werden sie später in Bedingungen (if, while) benutzen.

Wozu sind bitweise Operatoren gut?

- Sie sind **sehr schnell** – viel schneller als normale Rechenoperationen.
- Man kann damit **mehrere Zustände in einer einzigen Zahl** speichern (Bitmasken).
- Sie werden gebraucht, um **GPIO-Pins am Raspberry Pi** zu steuern.

Ein erstes Beispiel:

```
int zahl = 42;

if (zahl & 1) {
    printf("Die␣Zahl␣ist␣ungerade.\n");
}

...
```

2
Erste Schritte in C

3
Etwas Physik

Teil II

Die Programmiersprache C++

Teil III

Anhang